

GitLab CI/CD → AWS EC2

Classroom Demonstration Guide

Using the GitLab-hosted runner — NOT a self-managed EC2 runner

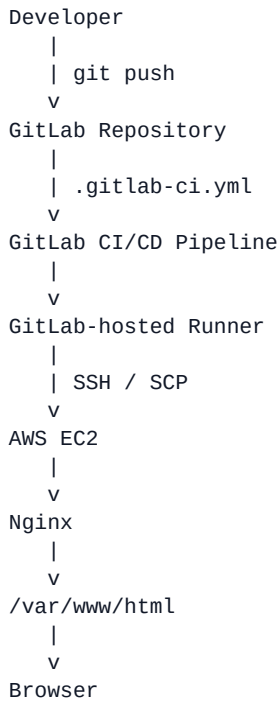
Step	Component	Purpose
1	Developer	git push
2	GitLab Repository	.gitlab-ci.yml
3	GitLab CI/CD	Pipeline + jobs
4	GitLab-hosted Runner	Runs CI/CD jobs
5	AWS EC2	Remote deployment target
6	Nginx	Serves /var/www/html
7	Browser	Displays application

Important: This guide intentionally uses GitLab-hosted runners. Do not install GitLab Runner on EC2 for this version. The hosted runner executes the job and connects to EC2 over SSH/SCP.

1. Demo Objective

Demonstrate a complete GitLab CI/CD workflow where a developer pushes an HTML/CSS application to GitLab, GitLab starts a pipeline, a GitLab-hosted runner executes the jobs, and the deployment job copies the application to AWS EC2 over SSH.

Final flow



Learning outcomes

- Understand how `.gitlab-ci.yml` defines stages and jobs.
- Understand that a runner executes the commands in each job.
- Understand the difference between a GitLab-hosted runner and a self-managed runner.
- Deploy an application from a hosted runner to AWS EC2 using SSH/SCP.
- Use GitLab CI/CD variables instead of hard-coding sensitive deployment information.

2. Prerequisites

- GitLab.com account and a GitLab project.
- AWS EC2 Ubuntu instance with Nginx installed.
- EC2 public IP address and SSH private key (.pem).
- EC2 Security Group allowing HTTP/80 and SSH/22 as required for the demo.
- Application files: index.html and style.css.
- GitLab-hosted runner available for the project.

3. Prepare AWS EC2

Install Nginx

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
sudo systemctl status nginx
```

Check the document root

```
ls -ld /var/www/html
ls -l /var/www/html/
```

Test Nginx

```
curl http://localhost
```

For this classroom demo, HTTP/80 must be reachable from your browser. SSH/22 must also be reachable from the GitLab-hosted runner for the deployment job.

4. Create the Demo Application

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DevOps CI/CD Demo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>DevOps CI/CD Demo</h1>
    <p>
      This application is deployed using
      <strong>GitLab CI/CD</strong>
      to an
      <strong>AWS EC2</strong> instance.
    </p>
    <div class="info">
      <p><strong>Source:</strong> GitLab</p>
      <p><strong>CI/CD:</strong> GitLab CI/CD</p>
      <p><strong>Server:</strong> AWS EC2</p>
      <p><strong>Web Server:</strong> Nginx</p>
    </div>
  </div>
</body>
</html>
```

style.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f4f6f8;
}

.container {
  width: 70%;
  margin: 100px auto;
  padding: 40px;
  background: white;
  text-align: center;
  border-radius: 10px;
  box-shadow: 0 4px 15px rgba(0,0,0,0.1);
}

h1 {
  margin-bottom: 20px;
}

.info {
  margin-top: 30px;
  padding: 20px;
  background: #f0f3f6;
  border-radius: 8px;
}
```

5. Prepare the GitLab Project

1. Create a blank GitLab project named devops-gitlab-demo.
2. Use the existing local project folder containing index.html and style.css, or clone the GitLab repository.
3. If the local folder still points to GitHub, change the remote to GitLab.

Change the remote if required

```
git remote -v  
git remote set-url origin https://gitlab.com/devops-training-group4/devops-gitlab-demo.git  
git remote -v
```

Push the application

```
git add .  
git commit -m "Prepare GitLab CI/CD demo"  
git branch -M main  
git push -u origin main
```

6. Configure GitLab CI/CD Variables

Go to **Project** → **Settings** → **CI/CD** → **Variables** and create these variables:

Key	Type	Value	Purpose
EC2_HOST	Variable	EC2 public IP	Target server
EC2_USER	Variable	ubuntu	SSH login user
EC2_SSH_KEY	File	Complete .pem private-key contents	SSH authentication

EC2_SSH_KEY must be a File-type variable. Paste the complete PEM key, including its beginning and ending lines. Do not commit the key to Git.

For this classroom setup, leave Protected OFF unless your main branch is protected, and make sure the variables are available to the branch running the pipeline.

7. Confirm the GitLab-hosted Runner

Do not install GitLab Runner on EC2 for this version. GitLab.com provides GitLab-hosted instance runners. The runner executes the CI job in a GitLab-managed environment, then the deployment job connects remotely to EC2.

- Open Project → Settings → CI/CD → Runners.
- Check the available/instance runners section.
- Confirm that a usable Linux GitLab-hosted runner is available and enabled.
- Keep the jobs untagged in this demo so they can use the default available hosted runner.

Account restriction note: If GitLab shows 'Identity verification is required in order to run CI jobs', the YAML is not the cause. Resolve the GitLab.com verification requirement before expecting the pipeline to execute.

8. Create .gitlab-ci.yml — Hosted Runner Version

This is the same hosted-runner configuration used before the deployment was blocked by identity verification. It uses an Ubuntu image, installs the SSH client, and deploys to EC2 with SCP/SSH.

```
stages:
  - test
  - deploy

test_application:
  stage: test
  image: ubuntu:24.04

  script:
    - test -f index.html
    - test -f style.css
    - echo "Application files found successfully"

deploy_application:
  stage: deploy
  image: ubuntu:24.04

  before_script:
    - apt-get update -y
    - apt-get install -y openssh-client
    - mkdir -p ~/.ssh
    - cp "$EC2_SSH_KEY" ~/.ssh/ec2_key
    - chmod 600 ~/.ssh/ec2_key
    - ssh-keyscan -H "$EC2_HOST" >> ~/.ssh/known_hosts

  script:
    - scp -i ~/.ssh/ec2_key index.html style.css "$EC2_USER@$EC2_HOST:/var/www/html/"
    - ssh -i ~/.ssh/ec2_key "$EC2_USER@$EC2_HOST" "ls -l /var/www/html/"

rules:
  - if: '$CI_COMMIT_BRANCH == "main"'
```

Important: This version is intentionally different from the self-managed-runner version. There is no local cp to /var/www/html because the hosted runner is not the EC2 server.

9. How the Pipeline Works

Stage 1 — test

```
test_application:
  stage: test
  image: ubuntu:24.04
  script:
    - test -f index.html
    - test -f style.css
    - echo "Application files found successfully"
```

- GitLab assigns the job to a hosted runner.
- The runner starts an Ubuntu 24.04 environment.
- The repository is available in the job workspace.
- The commands verify that index.html and style.css exist.

Stage 2 — deploy

- The deploy job starts after the test stage succeeds.
- The runner installs openssh-client.
- EC2_SSH_KEY is available as a temporary file.
- scp copies index.html and style.css to /var/www/html/ on EC2.
- ssh lists the deployed files to verify the deployment.
- The rules condition restricts deployment to the main branch.

10. Commit and Run the Pipeline

From the local project folder:

```
git add .gitlab-ci.yml
git commit -m "Add GitLab CI/CD deployment pipeline"
git push
```

Then in GitLab:

1. Open Build → Pipelines.
2. Open the new pipeline on the main branch.
3. Watch test_application execute first.
4. After it succeeds, watch deploy_application execute.
5. Open the deploy job and inspect the job log.

Expected result

```
Pipeline
|
+-- test_application      PASSED
|
+-- deploy_application    PASSED
|
v
AWS EC2 /var/www/html updated
```

11. Verify the Deployment

On the EC2 server:

```
ls -l /var/www/html/  
sudo systemctl status nginx
```

Open the EC2 public IP in a browser. The GitLab CI/CD demo page should appear.

Demonstrate continuous deployment

Change the heading in index.html to 'DevOps CI/CD Demo — Version 2', then run:

```
git add index.html  
git commit -m "Update application version"  
git push
```

GitLab creates another pipeline. After both jobs pass, refresh the browser and show the new version.

12. Troubleshooting

YAML invalid

Use Build → Pipeline editor → Validate. The YAML in Section 8 is the intended hosted-runner configuration.

Identity verification required

This is a GitLab.com account/project execution restriction. It is not a YAML or EC2 error.

Job stuck/pending

Check Settings → CI/CD → Runners and confirm a GitLab-hosted runner is available and enabled. Avoid adding tags that the runner does not have.

SSH connection timed out

Check EC2 Security Group TCP/22 and network access from GitLab-hosted runner infrastructure.

Permission denied copying files

Confirm the ubuntu SSH user can write to /var/www/html on EC2.

EC2_SSH_KEY errors

Ensure EC2_SSH_KEY is a File-type variable containing the complete PEM key. Do not echo/cat the private key in the job log.

Nginx still shows old page

Check /var/www/html/, confirm deploy job passed, and refresh the browser.

13. Hosted Runner vs Self-Managed Runner

Aspect	GitLab-hosted runner	Self-managed runner
Runner location	GitLab-managed temporary VM	Your infrastructure (for example EC2)
Install GitLab Runner on EC2	No	Yes
Deployment model here	Hosted runner → SSH/SCP → EC2	Runner can execute directly on EC2
Infrastructure management	GitLab manages runner	You manage runner
This guide uses	YES	NO

Teaching message: **In this demo, EC2 is the deployment target, not the GitLab runner.**

14. Security and Cleanup

- Never commit the EC2 private key to Git.
- Use GitLab CI/CD variables for sensitive deployment values.
- For production, avoid opening SSH (22) to 0.0.0.0/0; use a restricted network path or controlled source ranges.
- Use dedicated deployment credentials rather than personal keys for production.
- After the classroom demo, remove temporary broad security-group rules.
- Stop or terminate the EC2 instance when it is no longer required to avoid AWS charges.

15. Official References

GitLab-hosted runners

https://docs.gitlab.com/ci/runners/hosted_runners/

First GitLab CI/CD pipeline

https://docs.gitlab.com/ci/quick_start/

Using SSH keys with GitLab CI/CD

https://docs.gitlab.com/ci/jobs/ssh_keys/

CI/CD variables

<https://docs.gitlab.com/ci/variables/>

GitLab runners overview

<https://docs.gitlab.com/ci/runners/>

Prepared as a classroom lab guide for GitLab CI/CD → AWS EC2 using GitLab-hosted runners.